

ECEN 522: Final Project Report

By Luca Chierotti

Proximal Policy Optimization (PPO) Implementation

For this project, I implemented Proximal Policy Optimization (PPO) to train an agent to play Atari Pong. I chose PPO because it strikes a good balance between performance and stability, which is especially important when working with high-dimensional inputs like images. Unlike simpler policy gradient methods that can be unstable, PPO prevents the policy from changing too drastically between updates.

Environment Setup and Observation Processing

The agent was trained on the ALE/Pong-v5 environment with a frame skip of 4. The raw observations are RGB frames at 210×160 resolution, which contain a lot of unnecessary information—like the scoreboard and background—that can slow down learning. To handle this, I built a preprocessing pipeline. First, each frame is cropped to remove the scoreboard and margins, leaving a 160×160 gameplay region. Then it's converted to grayscale and downsampled to 80×80 to reduce computational cost. Finally, I apply an adaptive threshold to binarize the image, keeping only the high-contrast elements like the paddles, ball, and court lines.

Since Pong is dynamic, a single frame doesn't tell you which way the ball is moving. To capture this temporal information, I stack four consecutive processed frames together, creating a $4 \times 80 \times 80$ observation tensor. This frame-stacking technique is standard in Atari RL and lets the agent infer motion from pixels alone.

Reward Handling

During training, rewards are clipped to their sign (-1 , 0 , or $+1$). This prevents large reward values from dominating the loss function and helps stabilize learning, especially early on when the policy is still essentially random. During evaluation, I disable reward clipping so the reported scores reflect the actual game outcomes.

Network Architecture

The policy and value functions share a neural network backbone. This shared network has three convolutional layers followed by a fully connected layer with 512 hidden units, which extracts spatial features from the stacked frames.

From this shared representation, two separate heads branch off:

- **Policy head:** Outputs action probabilities (logits) over Pong's discrete action space
- **Value head:** Outputs a scalar estimate of how good the current state is

Sharing the early layers between actor and critic reduces the parameter count and improves sample efficiency, since the value function helps guide feature learning for the policy.

PPO Training Objective

Training uses on-policy rollouts collected from 4 parallel pong environments. After each rollout, I compute advantages using Generalized Advantage Estimation (GAE) with $\gamma=0.99$ and $\lambda=0.95$. The advantages are normalized to reduce variance and improve stability.

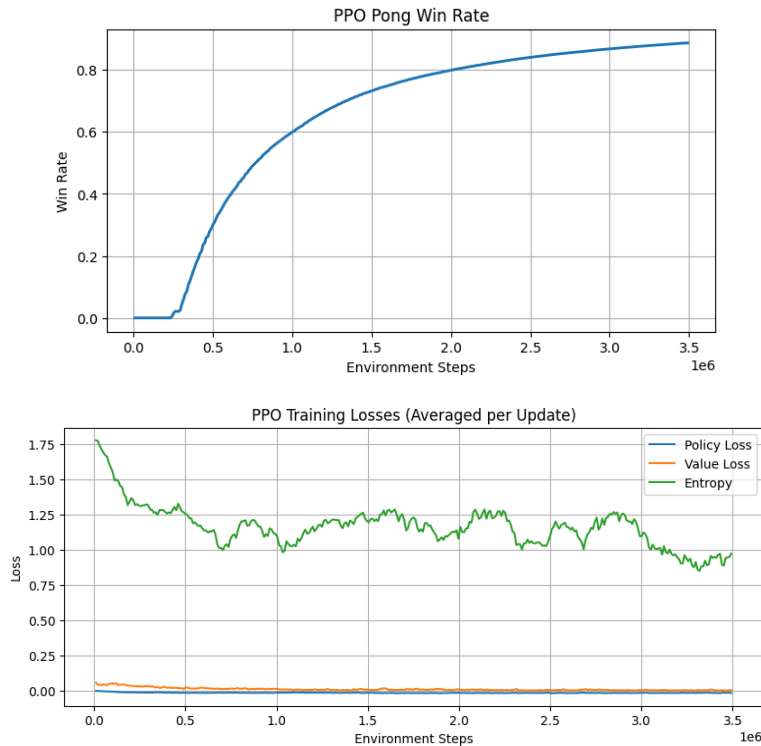
The core of PPO is its clipped surrogate objective, which limits how much the new policy can deviate from the old one by clipping the probability ratio. In addition to the policy loss, the objective includes a value function loss (to encourage accurate predictions) and an entropy bonus (to maintain exploration and prevent the policy from becoming too deterministic too early). I also apply gradient norm clipping for extra stability.

Training Procedure and Monitoring

Training was done using vectorized environments, which reduces correlation between samples compared to running a single environment. This improves both stability and sample efficiency. Throughout training, I tracked key metrics like win rate, policy loss, value loss, entropy, and episode length. Model checkpoints were saved periodically so I could analyze progress and keep the best performing models.

The agent's progression was clear: it started with near-random behavior, gradually learned to track the ball, began sustaining longer rallies, and eventually learned to score consistently against the built-in opponent. After about 500,000 steps, the win rate began climbing rapidly. By around 1 million steps, the agent was consistently winning most games. After around 2 million steps the learning rate slowed down significantly. The amount of steps it took to go from a win rate of 70% to 89% was 2 to 3.5 million steps. Due to the limited time available to complete this project, I decided that the 3.5 million step count was “good enough” to produce accurate results. Going up to a 90-95% win rate would have taken an additional million steps roughly, meaning an additional 30 min to 1 hour of training.

Training the PPO agent on Pong showed clear and stable improvement. The agent trained for approximately 3.5 million environment steps, which took about 1.5 hours on the T4 GPU on colab. PPO converged significantly faster than other methods I tried and needed much less hyperparameter tuning to achieve strong performance.



Win Rate Performance

The final training stats:

- **Final win rate:** approximately 88.6%
- **Episodes completed:** over 1,600
- **Final episode rewards:** frequently +21 (the maximum), indicating decisive victories

These results show that the PPO agent learned an effective and stable policy for Pong, capable of consistently beating the built-in opponent.

Loss and Entropy Behavior

The training loss curves support what I observed in the win rates. The policy loss stayed small and oscillated around zero, indicating controlled updates without excessive deviation from previous policies. The value loss decreased steadily and remained low, suggesting the value function was accurately estimating state values.

The entropy gradually decreased over time, which is expected as the agent transitions from exploration to exploitation. Importantly, entropy didn't collapse prematurely—the entropy bonus successfully maintained exploration during the early and middle stages of training. Even late in training, the policy retained some randomness, which helps prevent overfitting to specific patterns.

Extended Evaluation over 100 Episodes

To make sure the strong performance I saw during training wasn't just a fluke, I ran a thorough evaluation of the final PPO model over 100 consecutive episodes. For this evaluation, I used deterministic action selection (always picking the best action) and disabled reward clipping to get the true game scores.

The results were remarkable: the agent achieved a 100% win rate across all 100 games, with an average score of exactly +21.0 and zero variance. Since Pong ends when one player reaches 21 points, this means the agent won every single game by the maximum possible margin—essentially shutting out the opponent in every match.

What's even more interesting is how consistent the gameplay was. Each episode took an average of 1,977 environment steps, with no variation at all across the 100 games. This consistency shows the policy learned a stable, repeatable strategy that produces nearly identical game trajectories every time. With a frame skip of 4, that's about 7,900 actual game frames per episode, or roughly two minutes of gameplay per match.

The complete lack of variance in both scores and episode lengths tells us something important: the PPO agent converged to what's essentially a deterministic optimal policy. It's not winning by getting lucky or waiting for the opponent to make mistakes—it's consistently tracking the ball, positioning the paddle correctly, and methodically winning each game. This demonstrates that the policy genuinely learned the task and isn't sensitive to starting positions or random chance.

Deep Q-Network (DQN) Implementation

To have something to compare against PPO, I also implemented a Deep Q-Network (DQN) agent for Atari Pong. DQN is one of the foundational algorithms in deep RL for Atari games, so it makes for a good baseline. It also provides an interesting contrast to policy gradient methods in terms of how it learns, how stable the training is, and how efficiently it uses data.

Environment, Preprocessing, and Network Architecture

The DQN agent uses the exact same setup as PPO—same ALE/Pong-v5 environment with frame skip of 4 and deterministic dynamics. I kept the preprocessing pipeline identical too, to make sure any performance differences were due to the algorithm itself, not the input processing. That means the same pipeline: crop the frame, convert to grayscale, downsample to 80×80, binarize it, and stack four consecutive frames together so the network can figure out how the ball is moving.

Like with PPO, rewards are clipped to their sign (-1, 0, +1) during training to keep the temporal-difference updates stable, which is pretty standard practice for Atari DQN.

The DQN model uses a convolutional neural network similar to the original Atari DQN architecture. It has several convolutional layers followed by fully connected layers that output a Q-value for each possible action. Given the stacked frames, the network estimates the expected future reward for each action, and the agent picks actions using ϵ -greedy exploration (mostly taking the best action, but sometimes exploring randomly).

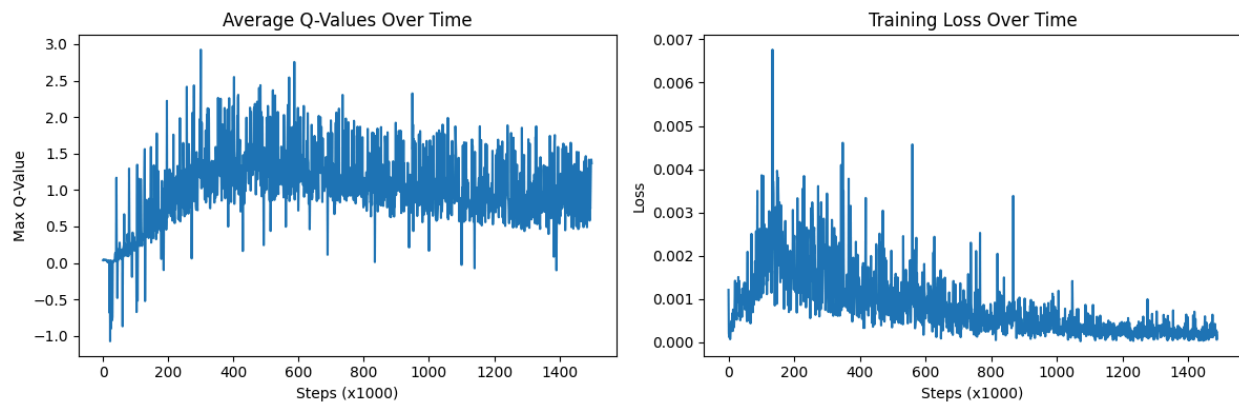
Training

During training, the agent explores using ϵ -greedy action selection, where ϵ (the probability of taking a random action) gradually decreases over time. This encourages exploration early on and exploitation later. The network learns by minimizing the mean squared error between its predicted Q-values and the target Q-values computed using the target network.

The DQN agent was trained on Atari Pong for approximately 1.5 million environment steps, which took around **36 hours** of actual training time. That's a stark contrast to PPO—much longer to reach competitive performance. Despite the extended training period, the DQN agent did eventually learn a strong policy that could consistently beat the built-in Pong opponent, though the journey there was notably bumpier.

Looking at the training metrics, you can see how DQN learned over time. The average Q-values plot shows a gradual increase during early training, which is a good sign—it means the network was learning to assign higher expected rewards to good states and actions. Over time, the Q-values stabilize, though they remain pretty noisy throughout. This reflects the inherent variance in temporal-difference learning and how sensitive Q-value estimates can be to small policy changes.

The training loss decreases steadily overall, but there are some notable spikes early on. These spikes happen when the replay buffer contains really diverse transitions or when the policy shifts significantly due to exploration. As training continues and exploration decreases (ϵ decays), the loss curve smooths out and converges to small values, showing that the Q-network learned a relatively stable value function. The training curves indicate successful convergence, but with noticeably more variance and slower stabilization than what I saw with PPO.



Evaluation Performance Compared to PPO

After training, I evaluated the DQN agent over 100 episodes using deterministic action selection. Here's what I found:

- **Wins / Losses:** 96 / 4
- **Win rate:** 96.0%
- **Average score:** +11.76
- **Average steps per episode:** 3,762.8

A 96% win rate is impressive—it shows the DQN agent definitely learned an effective policy and can consistently outperform the Pong opponent. However, the average score of +11.76 tells a different story than PPO's perfect +21.0. Most DQN wins weren't blowouts; instead, games often ended with narrower margins. The agent could reliably win, but it didn't dominate the way PPO did. The episode length is particularly interesting. At roughly 3,763 steps per game, DQN episodes took almost twice as long as PPO's 1,977 steps. This suggests games involved longer rallies and slower point resolution. While this shows the agent has reasonable ball tracking and paddle control, it also hints at less precise positioning or slower closing behavior compared to PPO's more aggressive strategy.

	DQN	PPO
Training time	~36 hours	~1.5 hours
Win rate	96%	100%
Average score	+11.76	+21.0
Episode length	3,763 steps	1,977 steps

While DQN ultimately achieved high win rates, it required substantially more training time and showed much noisier learning dynamics. PPO reached near-optimal performance in about 1.5 hours, achieved a perfect win rate, and consistently won by the maximum margin. PPO also converged to a more deterministic strategy—evidenced by zero variance in both scores and episode lengths during evaluation.

These results highlight a fundamental difference between value-based and policy-gradient methods for this task. DQN is certainly capable of learning strong Pong policies, but PPO offers dramatically faster convergence, smoother training, and more consistent final performance. For Atari Pong specifically, the on-policy nature of PPO combined with parallel environments and advantage estimation proved to be a winning combination.